

Chapter 13 Modeling Computation

Chapter Summary

- Languages and Grammars
- Finite-State Machines with Output
- Finite-State Machines with No output
- Language Recognition
- Turing Machines

Section 13.1 Languages and Grammars

Section Summary

- Phrases-Structure Grammars
- Types of Phrases-Structure grammars
- Derivation Trees
- Backus-Naur Form

Introduction

Syntax: Form of a sentence

Semantics: Meaning of a sentence.

Grammars

The rules that specify the **syntactically** correct sentences.

Natural language: Like English, Chinese and so on. They are complex.

Formal Language: Well-defined rules of syntax.

Examples of Grammar:

- A sentence is made up of a noun phrase followed by a verb phrase
- a verb is *eat*
- a noun is *rabbit*
- $\vdots$

Phrase-Structure Grammars

- **Vocabulary** set V is a **finite, nonempty set of elements** called **symbols**.

- **Word** over V is a string of finite length of elements in V .
- **Empty string or null string** contains no symbols, denoted by λ
- The set of **all words over V** is denoted by V^* . A **language** over V is a subset of V^* .
- The elements of V that can **not be replaced** are called **terminals**; others are called **nonterminal**.
- The rules that specify when we can replace a string with another string in V^* are called **productions** of the grammar, denoted by $z_0 \rightarrow z_1$ that z_0 can be replaced by z_1 within a string.

A **phrase-structure grammar** $G = (V, T, S, P)$ consist a vocabulary V , a terminal subset T of V , a start symbol S from V , a finite set of productions P . Nonterminal symbol set $N = V - T$. Every production in P must contains at least one nonterminal on its left side.

Example:

Let $G = (V, T, S, P)$ where $V = \{a, b, A, B, S\}$, $T = \{a, b\}$, S is the start symbol and $P = \{S \rightarrow Aba, A \rightarrow BB, B \rightarrow ab, AB \rightarrow b\}$.

Step 1: Replace S by Aba ;

Step 2: Replace A by BB ;

Step 3: Replace B by ab .

Then all symbols are terminals thus the result is $ababba$.

To summarize, continue to replace the nonterminal to terminals.

Derivations

Let $G = (V, T, S, P)$ be a phrase-structure grammar. $w_0 = lz_0r$, $w_1 = lz_1r$ are strings over V . If $z_0 \rightarrow z_1$ is a production of G , we say w_1 is **directly derivable** from w_0 , denoted by $w_0 \Rightarrow w_1$.

If $w_0 \Rightarrow w_1, w_1 \Rightarrow w_2, \dots, w_{n-1} \Rightarrow w_n$, then w_n is **derivable** from w_0 and write $w_0 \Rightarrow w_n$. The sequence of steps used to obtain w_n from w_0 is called a **derivation**.

Example: $Aaba$ is directly derivable from ABa . $ababba$ is derivable from Aba .

Language Generation

Let $G = (V, T, S, P)$ be a phrase-structure grammar. The **language generated** by G , denoted by $L(G)$ is the set of all **strings or terminals** that **derivable from the starting state S** .

For example, $G = (\{S, A, a, b\}, \{a, b\}, S, \{S \rightarrow aA, S \rightarrow b, A \rightarrow aa\})$, then $L(G) = \{b, aaa\}$ (all possible results)

Types of Phrases Structure

TABLE 1 Types of Grammars.	
Type	Restrictions on Productions $w_1 \rightarrow w_2$
0	No restrictions
1	$w_1 = lAr$ and $w_2 = lwr$, where $A \in N, l, r, w \in (N \cup T)^*$ and $w \neq \lambda$; or $w_1 = S$ and $w_2 = \lambda$ as long as S is not on the right-hand side of another production
2	$w_1 = A$, where A is a nonterminal symbol
3	$w_1 = A$ and $w_2 = aB$ or $w_2 = a$, where $A \in N, B \in N$, and $a \in T$; or $w_1 = S$ and $w_2 = \lambda$

Type 1 grammars are called **context-sensitive grammars**, corresponding language is called **context-sensitive grammars**.

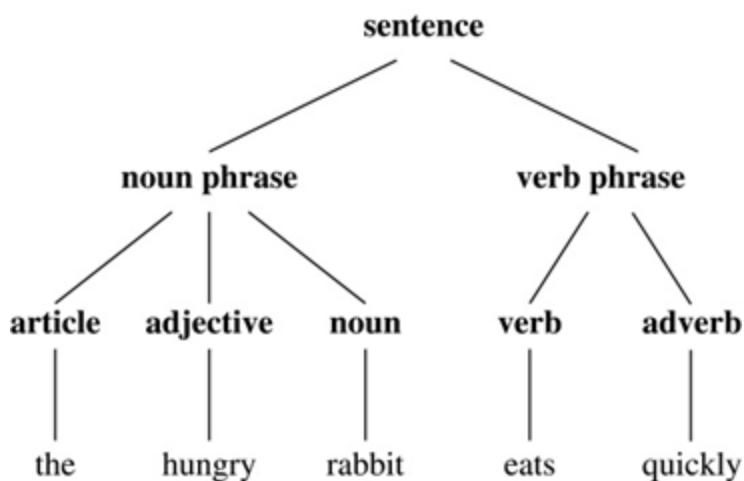
Type 2 grammars are called **context-free grammars**, corresponding language is called **context-free language**.

Type 3 grammars are called **regular grammars**, corresponding language is called **regular language**.

Derivation Trees

We can represent a derivation in the language generated by **context-free grammars** by an ordered rooted tree, called a **derivation or parse tree**.

- root is start symbol
- internal vertices represent nonterminal symbols
- leaves represent terminal symbols
- for $A \rightarrow w$, where w is a word, A has children vertices represent each symbol in w in order from left to right.



Backus-Naur Form

BNF is sometimes used to **specify type 2 grammar**. It is often used to specify the syntactic rules of computer languages.

- Use symbol $::=$ instead of $\rightarrow$
- All nonterminal symbols are enclosed by $\langle \rangle$
- Right-hand side of productions are separated by bars

For example, the production $A \rightarrow Aa, A \rightarrow a, A \rightarrow AB$ are written as

$$\langle A \rangle ::= \langle A \rangle a | a | \langle A \rangle \langle B \rangle$$

BNF and ALGOL 60

In programming language ALGOL 60, identifiers consist a string of alphanumeric characters and must begin with a letter.

Allowable BNF description:

$$\begin{aligned}\langle identifier \rangle &::= \langle letter \rangle | \langle identifier \rangle \langle letter \rangle | \langle identifier \rangle \langle digit \rangle \\ \langle letter \rangle &::= a | b | \dots | y | z \\ \langle digit \rangle &::= 0 | 1 | \dots | 9\end{aligned}$$

Section 13.2 Finite-State Machines with Output

Section Summary

- Finite-State Machines (FSMs) with Outputs
- Types of Finite-State Machines with outputs

Introduction

Many kinds of machines including computers can be modeled using a structure called **finite-state machine** (or finite automation), which consist a **finite set of states**, a **designated start state**, an **input alphabet**, a **transition function** that assign a next state to every (start, input) pair.

An Example of a Finite-State Machine with Output

A vending machine accepts 5 cents, 10 cents, 25 cents. When 30 cents or more has been deposited, the machine returns the amount over 30 cents. The customer can then press an orange button to receive orange juice and red button to get apple juice.

The machine can be in any of the **state** $s_i, i = 0, \dots, 6$ where s_i is the state where the machine has received $5i$ cents. The **start state** is s_0 . **Possible input** are 5 cents, 10 cents, 25 cents, the

orange button and the red button. The transition function table:

TABLE 1 State Table for a Vending Machine.										
	Next State					Output				
	<i>Input</i>					<i>Input</i>				
<i>State</i>	5	10	25	<i>O</i>	<i>R</i>	5	10	25	<i>O</i>	<i>R</i>
s_0	s_1	s_2	s_5	s_0	s_0	n	n	n	n	n
s_1	s_2	s_3	s_6	s_1	s_1	n	n	n	n	n
s_2	s_3	s_4	s_6	s_2	s_2	n	n	5	n	n
s_3	s_4	s_5	s_6	s_3	s_3	n	n	10	n	n
s_4	s_5	s_6	s_6	s_4	s_4	n	n	15	n	n
s_5	s_6	s_6	s_6	s_5	s_5	n	5	20	n	n
s_6	s_6	s_6	s_6	s_0	s_0	5	10	25	OJ	AJ

FSMs with Outputs

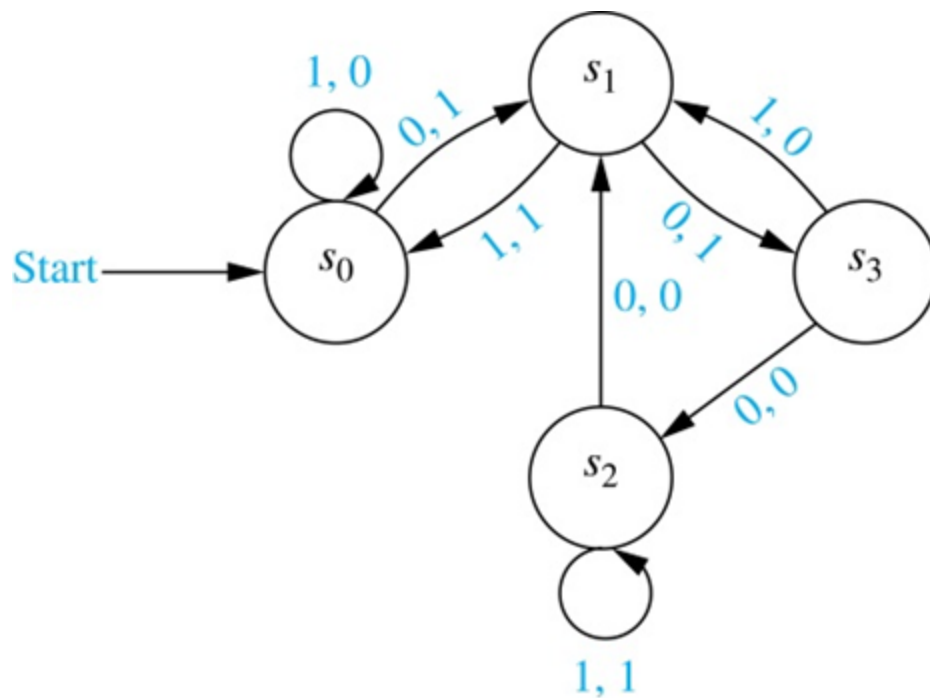
A finite-state machine $M = (S, I, O, f, g, s_0)$ consists of a finite set S of states, a finite input alphabet I , a finite output alphabet O , a transition function f that assigns to each state and input pair a new state, an output function g that assigns to each state and input an output and an initial state s_0 .

Two methods to represent f and g :

- State table

TABLE 2				
<i>State</i>	<i>f</i>		<i>g</i>	
	<i>Input</i>		<i>Input</i>	
	0	1	0	1
s_0	s_1	s_0	1	0
s_1	s_3	s_0	1	1
s_2	s_1	s_2	0	1
s_3	s_2	s_1	0	0

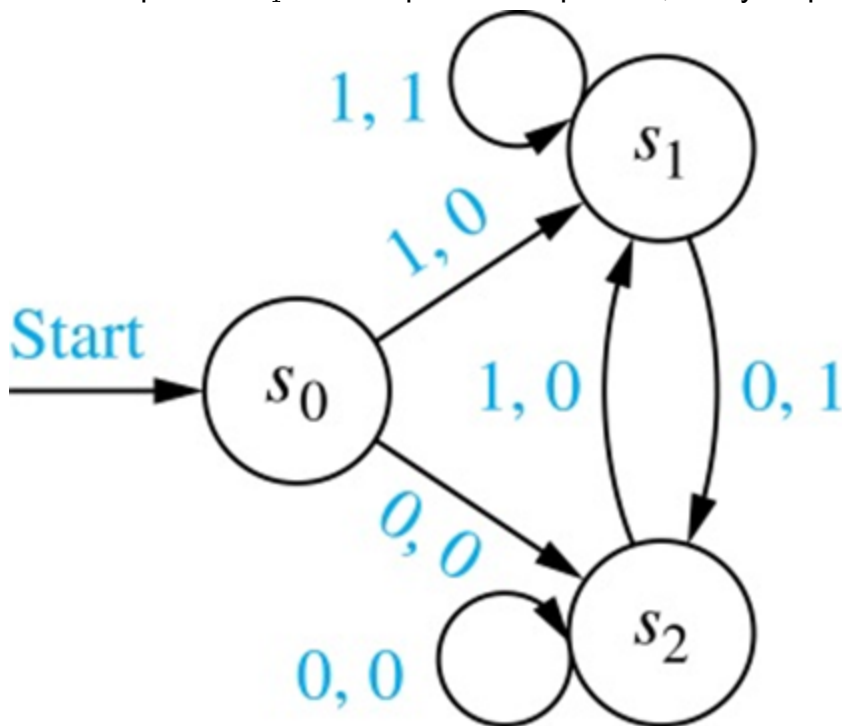
- State diagram: a directed graph with labeled edges. State is represented by circle, arrows labeled with input and output pair represent transitions.



Unit-delay Machine

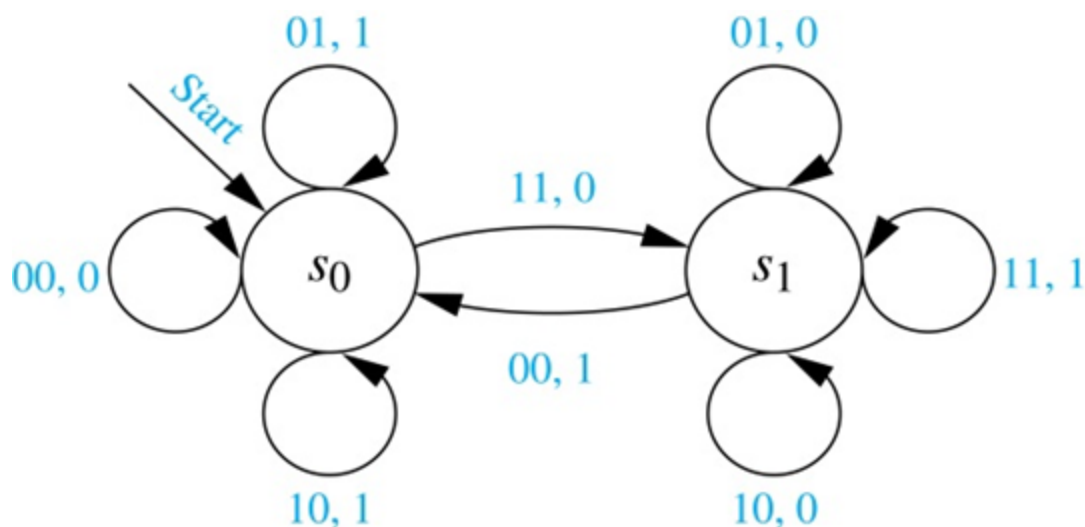
Unit-delay machine produces as output the input string delayed by a specified amount of time. i.e., padded with an initial string of 0s.

A delay machine can be constructed that has 0 or 1 as possible inputs. It start at state s_0 , transition from s_0 produces an output of 0. s_1 receive input 0 and output 1 to state s_2 . s_2 receive 1 and output 0 to s_1 . Thus input 101 output 010, delay a specified amount of time.



Addition Machine

Add two positive integers using their **binary expansions** $(x_n \cdots x_1 x_0)_2$ and $(y_n \cdots y_1 y_0)_2$. Every addition receive input x_i, y_i and a carry bit c_{i-1} and output a sum bit z_i and a carry bit c_i . Repeat until the most significant digit and the last carry c_n is the MSD of result z_{n+1} .



s_0 represent the carry of last step is 0(initial state), s_1 represent the carry of last step is 1.

Section 16.3 Finite-State Machines with No Output

Section Summary

- Set of Strings
- Finite-State Automata
- Language Recognition by Finite-State Machines
- Designing Finite-State Automata
- Equivalent Finite-State Automata
- Nondeterministic Finite-State Automata

Set of Strings

FSMs without output but with some states designated as **accepting states** are designed for **recognizing languages**.

The **concatenation** of A and B where A, B are subset of V^* , denoted by AB is the **set of all strings of the form ab** where $a \in A, b \in B$.

For example, $A = \{0, 11\}, B = \{1, 10, 110\}$, then $AB = \{01, 010, 0110, 111, 1110, 11110\}$, $BA = \{10, 111, 100, 1011, 1100, 11011\}$.

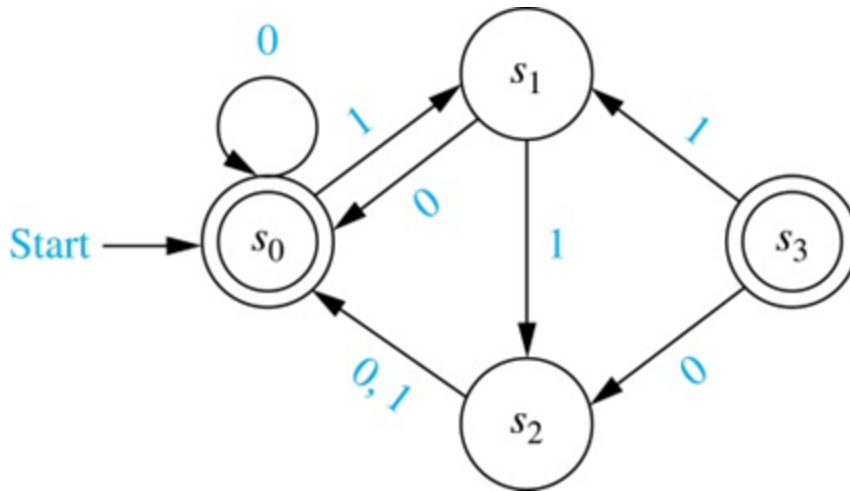
If $A \subseteq V^*$, the **Kleene closure** of A denoted by A^* is the set consisting of arbitrarily long strings of elements of A , that is, $A^* = \bigcup_{k=0}^{\infty} A^k$. (A^k is the string with length k).

Finite-State Automata(FSA)

It is finite-state machine without output but with accepting states.

A finite-state automaton $M = (S, I, f, s_0, F)$ consists of a finite set S of states, a finite input alphabet I , a transition function f that assigns a next state to every pair of state and input, an initial or start state s_0 and a subset F of S consisting of final(or accepting) states.

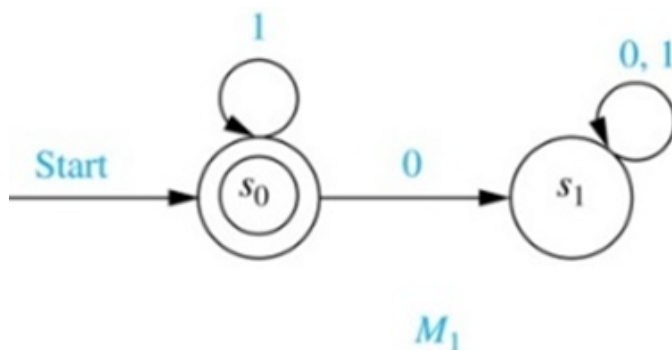
It also can be represented by state table or state diagrams, in which final states are indicated with a double circle.



Language Recognition by FSAs

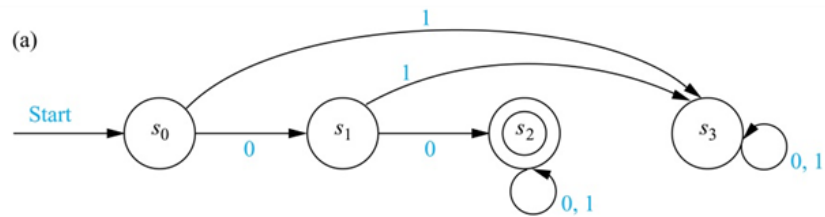
A string x is said to be recognized(or accepted) by $M = (S, I, f, s_0, F)$ if it takes the initial state s_0 to final state. The language recognized(or accepted) by M , denoted by $L(M)$ is the set of all strings that are recognized by M . Two finite-state automata are called equivalent if they recognize the same language.

For example,

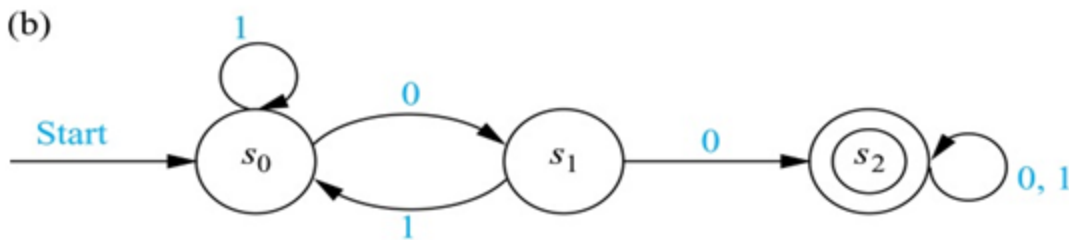


$$L(M_1) = \{1^n | n = 0, 1, \dots\}$$

Construct a FSA that recognizes the set of bit strings that begin with two 0s:



Constructing a FSA that recognizes the set of bit strings that contains two consecutive 0s:



NDFSA

Nondeterministic finite-state automaton $M = (S, I, f, s_0, F)$ consists of a finite set S of states, a finite input alphabet I , a transition function f that assigns **a set of states** to every pair of state and input, an initial or start state s_0 , and a subset F of S consisting of final (or accepting) state. The distinction between before DFSA and NDFSA is that **the transition function**. The result of transition function of NDFSA is **not unique state but a set of states**.

A string is recognized by NDFS if and only if there exist at least transition path from s_0 to a accepting state in F .

It also can be represented by state diagram and state table.

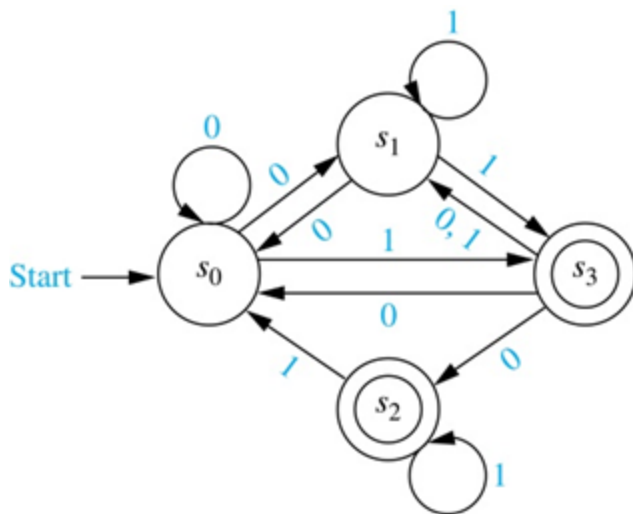


TABLE 2

State	f	
	Input	
	0	1
s_0	s_0, s_1	s_3
s_1	s_0	s_1, s_3
s_2		s_0, s_2
s_3	s_0, s_1, s_2	s_1

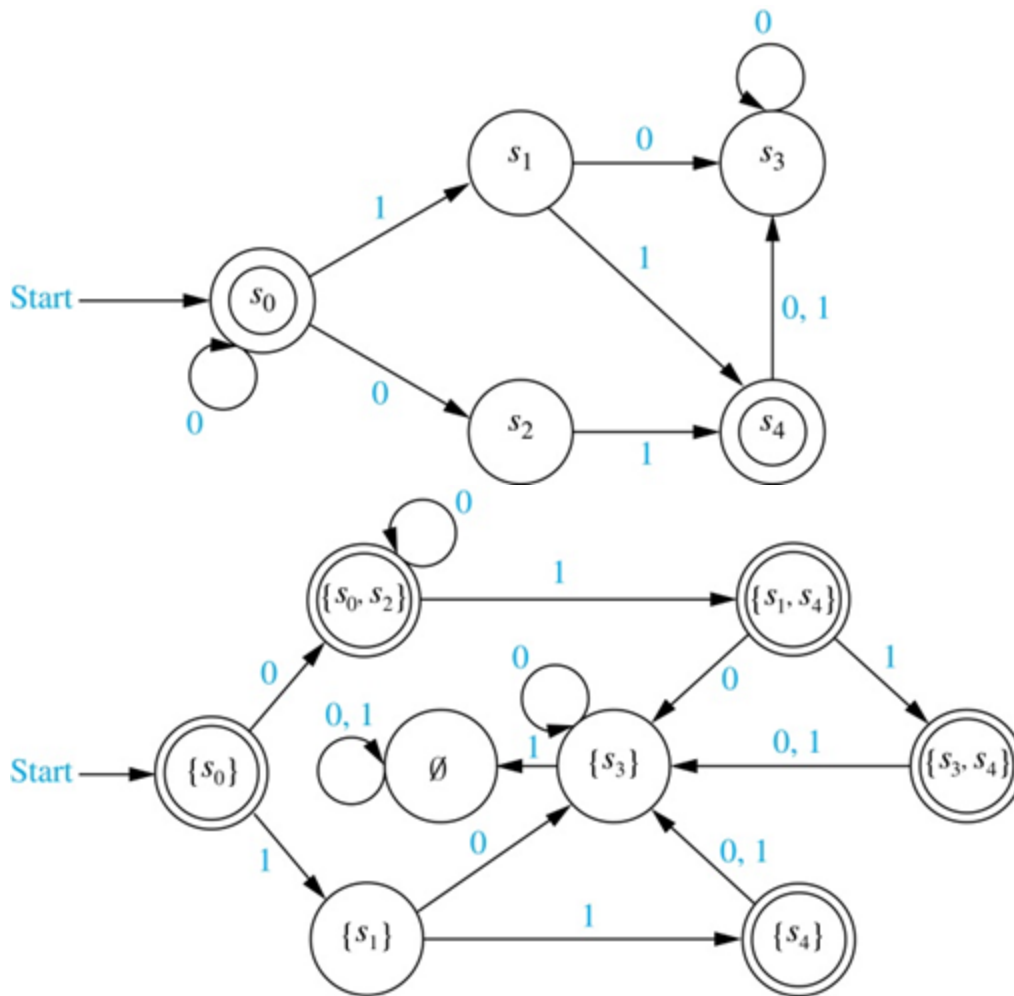
Finding a DFSA Equivalent to a NDFSA

For every NDFSA there is an equivalent DFSA.

We construct the DFSA M_1 equivalent to NDFSA M_0 so that

- Each state in M_1 is made up of a set of states in M_0 . (For example, the set states in M_0 is $\{s_0, s_1\}$, then one state in M_1 is $\{s_0, s_1\}$)
- Start symbol of M_1 is $\{s_0\}$.
- Input set of M_1 is the same as input set of M_0 .
- For a state $\{s_{i_1}, \dots, s_{i_k}\}$ in M_1 , the input symbol takes this state to the union of the sets of next states for the elements of this set. That is the union of the sets $f(s_{i_1}, x), \dots, f(s_{i_k}, x)$.
- The final states of M_1 are the sets that contain final state of M_0 .

Example:



Section 13.4 Language Recognition

Section Summary

- Regular Expressions
- Kleene's Theorem
- Regular Sets and Regular Grammars
- More Powerful Types of Machines

Regular Expressions

Regular expressions over a set I is defined recursively by:

- the symbol $\emptyset$; λ ; x is a regular expression where $x \in I$.
- The symbol (AB) , $(A \cup B)$ and A^* are regular expression whenever A and B are regular expressions.

Sets represented by regular expressions are called regular sets.

Example: explain the strings in the regular sets specified by the regular expressions 10^* , $(10)^*$, $0 \cup 01$, $0(0 \cup 1)^*$, $(0^*1)^*$

TABLE 1	
Expression	Strings
10^*	a 1 followed by any number of 0s (including no zeros)
$(10)^*$	any number of copies of 10 (including the null string)
$0 \cup 01$	the string 0 or the string 01
$0(0 \cup 1)^*$	any string beginning with 0
$(0^*1)^*$	any string not ending with 0

Finite-State Automata, Regular Sets and Regular Grammars

Kleene's Theorem:

A set is regular if and only if it is recognized by a FSA.

A set is generated by a regular grammar if and only if it is a regular set.

A Set Not Recognized by a FSA

The set $L = \{0^n 1^n \mid n = 0, 1, \dots\}$ is not regular. It can be proved by contradiction. To show there is no NDFSA recognizing it.

The conclusion is that the number of states in FSA is finite.

More Powerful Types of Machines

- Pushdown Automaton(PDA)
- Linear Bounded Automaton(LBA)
- Turing Machine(TM)

Section 13.5 Turing Machines

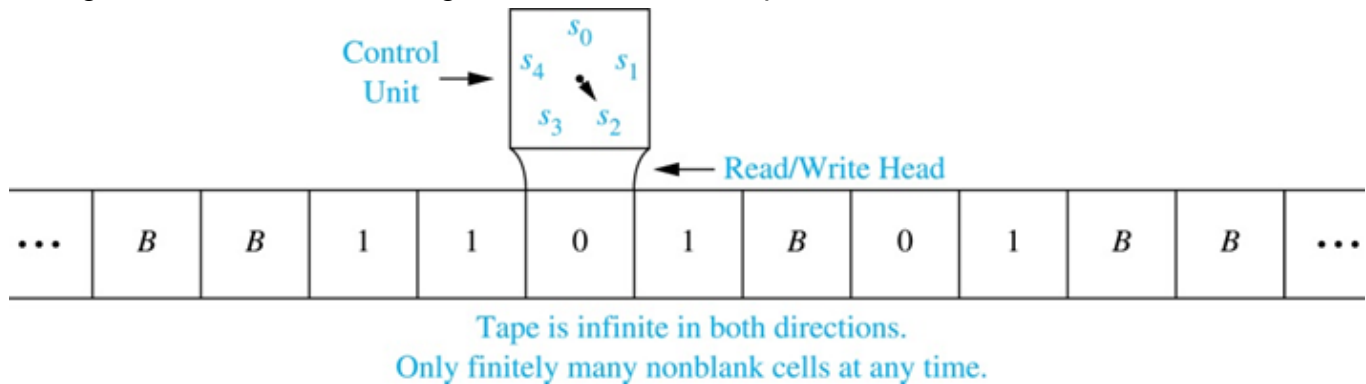
Section Summary

- Definition of Turing Machines
- Using Turing Machines to Recognize Sets
- Computing Functions with Turing Machines
- Different Types of Turing Machines
- The Church-Turing Thesis
- Computational Complexity, Computability and Decidability

Introduction

Informally, a Turing machine consists of a control unit together with a tape that infinite in both directions, which is divided into cells.

Turing machines are the most general models of computation.



Definition of Turing Machine (TM)

A Turing machine $T = (S, I, f, s_0)$ consists of a finite set S of states, an alphabet I containing the blank symbol B , a partial function f from $S \times I$ to $S \times I \times \{R, L\}$ (direction) and a starting state s_0 .

f is partial function. i.e., for some (state, symbol) pairs it may be undefined. For defined, there is a unique (state, symbol, direction) triple associated to this pair.

The corresponding five-tuples (s, x, s', x', d) are called transition rules of the machine.

At each step, control unit reads current tape symbol x , then for current state s and the partial function $f(s, x) = (s', x', d)$, the control unit:

- enter the state s'
- write the symbol x' in current cell and erase x
- Move right or left one cell depends on d .

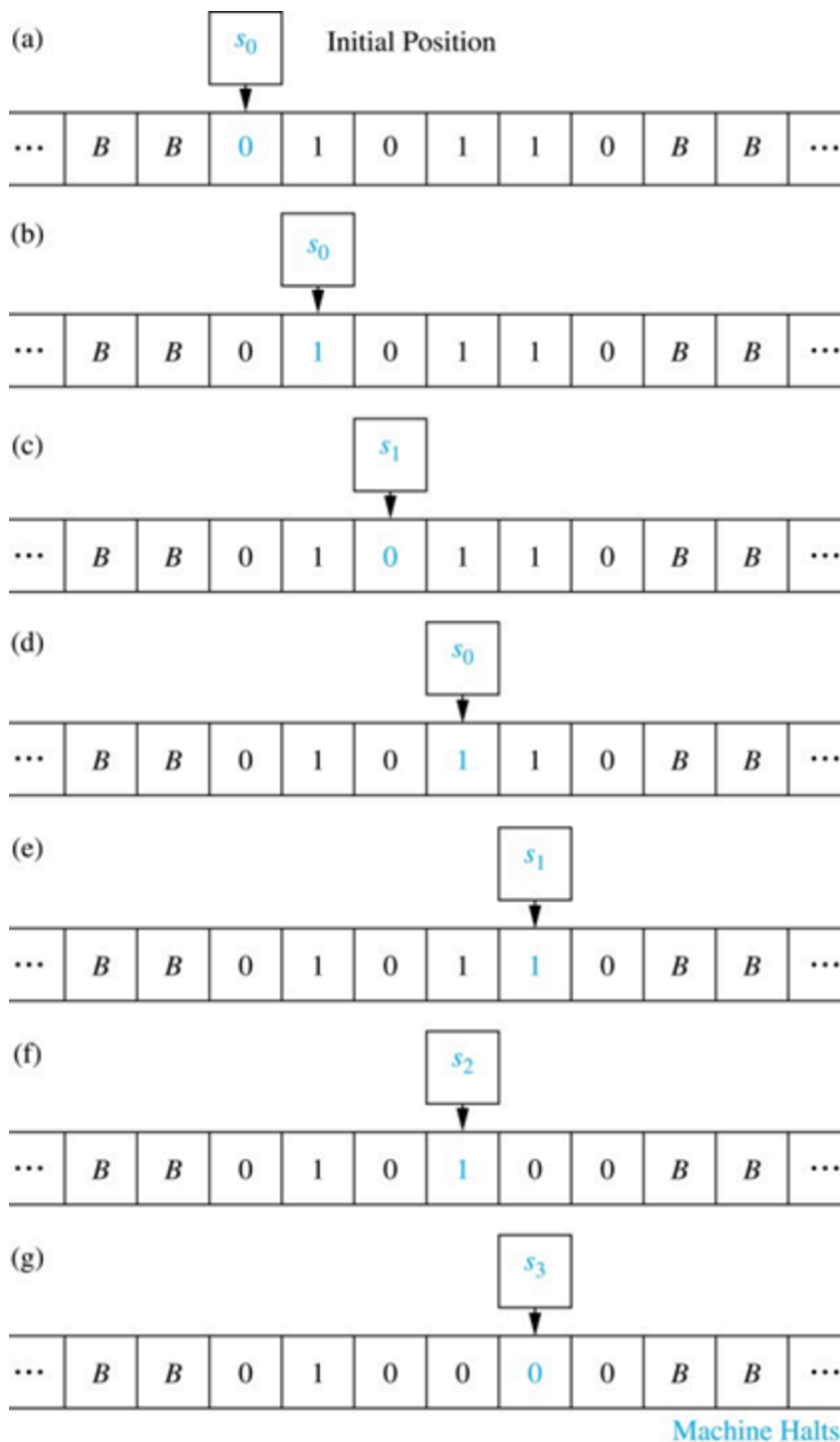
When the partial function is undefined, the machine will halt. The initial state is positioned over the leftmost nonblank symbol on the tape. This is the initial position of the machine.

For nondeterministic Turing machine (NDTM), the restriction that no two transition rules begin with the same pair (s, x) is eliminated.

A TM in Operations

Example: The TM T is defined by seven five-tuples

$(s_0, 0, s_0, 0, R), (s_0, 1, s_1, 1, R), (s_0, B, s_3, B, R), (s_1, 0, s_0, 0, R),$
 $(s_1, 1, s_2, 0, L), (s_1, B, s_3, B, R), (s_2, 1, s_3, 0, R)$



Using TM to Recognize Sets

Let V be a subset of alphabet I . $T = (S, I, f, s_0)$ recognizes a string x in V^* if and only if when x is written on the tape T starting in the initial position and halts in a final state.

T is said to recognize a subset A of V^* if x is recognized by T if and only if $x \in A$. (We can use symbols not in V)

For the set $\{0^n 1^n | n \geq 1\}$ that cannot be recognized by DFA, TM can recognize this set.
 $V = \{0, 1\}$, $I = \{0, 1, M\}$ where M is a marker. TM successively replaces a 0 at leftmost

position of string with M and a 1 at the rightmost position of the string with M . Terminate if and only if the string consists of same number of 0 and 1.

Computing Functions with TMs

Suppose T when given string x as input, halts with the string y on its tape. We can define $T(x) = y$.

We use **unary representation of integers** to consider a TM as a computer from the set k -tuples of nonnegative integers to the set of nonnegative integers. And we use $*$ to separate different integers in k -tuple (n is represented by $n + 1$ 1s, for example, 5 is 11111, (2, 0, 2, 5) is 111 * 1 * 111 * 11111)

To compute $f(n_1, n_2) = n_1 + n_2$,

- transfer the pair by unary representation
- Start at the left most 1 of the input string and proceeds to erase this 1.
- If the next is $*$, $n_1 = 0$ and then replace the asterisk with blank and halts
- Otherwise, erase next 1 and pass over the remaining 1s until encountering $*$ and then replace it by 1.

The Church-Turing Thesis

Any problem that can be solved with an **effective algorithm** (informal and imprecise concept), there is a TM that can solve this problem (formal and precise concept).

Decidability and Complexity

- **Decision problem** asks whether statements from a particular statements are true. (Like halting problem)
- If there is an **effective algorithm** decides the decision problem, the problem is **solvable or decidable**. (halting problem is unsolvable)
- A function that can be computed by TM is called **computable**. (Busy beaver function is incomputable)

The Classes P and NP

If a problem can be solved by DTM in polynomial time in terms of the size of input, it is in **the class of polynomial-time problems**, or **Class P**.

If a problem can be solved by NDTM in polynomial time in terms of the size of input, it is in **the class of nondeterministic polynomial-time problems**, or **Class NP**.

Problems in P are called **tractable** while not in P are called **intractable**.

Since DTM can also be considered to be a NDTM, $P \subseteq NP$. (problem: whether $P = NP$)

NP-complete problems:

- It is in class NP
- If the existence of a polynomial-time algorithm for solving it, there exist a polynomial-time algorithm for every problem in NP.